


Inteligência Artificial

Busca Competitiva

Setembro, 2026




Motivação

Busca clássica

O ambiente é passivo: o agente escolhe ações para chegar a um objetivo.

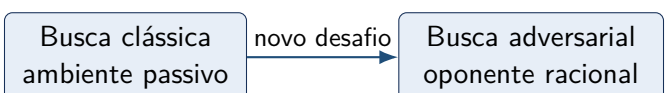
Motivação

Busca clássica

O ambiente é passivo: o agente escolhe ações para chegar a um objetivo.

Busca competitiva

O ambiente contém outro agente que também escolhe ações para maximizar sua utilidade.



E se o ambiente também estiver tentando impedir você?

Busca Competitiva

- Busca Competitiva é um paradigma de busca onde múltiplos agentes competem para alcançar um objetivo.
- Cada agente tenta maximizar sua própria utilidade, muitas vezes em detrimento dos outros.
- Exemplos incluem jogos de tabuleiro, leilões e mercados financeiros.

Busca Competitiva

- Busca Competitiva é um paradigma de busca onde múltiplos agentes competem para alcançar um objetivo.
- Cada agente tenta maximizar sua própria utilidade, muitas vezes em detrimento dos outros.
- Exemplos incluem jogos de tabuleiro, leilões e mercados financeiros.

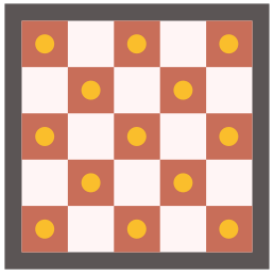


Figure: Jogo de damas.

Busca Competitiva

- Busca Competitiva é um paradigma de busca onde múltiplos agentes competem para alcançar um objetivo.
- Cada agente tenta maximizar sua própria utilidade, muitas vezes em detrimento dos outros.
- Exemplos incluem jogos de tabuleiro, leilões e mercados financeiros.

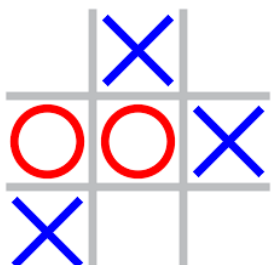


Figure: Jogo da velha.

Busca Competitiva - História

- A Busca Competitiva tem suas raízes em teorias de jogos / economia.
- Modelos de agentes competindo por recursos limitados foram estudados desde a década de 1970.
- Com o avanço da computação, simulações de busca competitiva tornaram-se populares em IA.

Busca Competitiva - História

- Damas
  - 1950 - Arthur Samuel desenvolve um programa de damas que aprende com a experiência.
  - 1994 - Chinook vence o campeão mundial de damas, Marion Tinsley.
  - 2007 - Chinook é provado como um jogo resolvido, garantindo pelo menos um empate com jogo perfeito.

Busca Competitiva - História

- Damas
  - 1950 - Arthur Samuel desenvolve um programa de damas que aprende com a experiência.
  - 1994 - Chinook vence o campeão mundial de damas, Marion Tinsley.
  - 2007 - Chinook é provado como um jogo resolvido, garantindo pelo menos um empate com jogo perfeito.
- Xadrez
  - 1997 - Deep Blue da IBM derrota o campeão mundial Garry Kasparov.
  - 2006 - Deep Fritz vence o campeão mundial Vladimir Kramnik.
  - 2017 - AlphaZero da DeepMind aprende xadrez do zero e vence o Stockfish.

Busca Competitiva - História

- Damas
  - 1950 - Arthur Samuel desenvolve um programa de damas que aprende com a experiência.
  - 1994 - Chinook vence o campeão mundial de damas, Marion Tinsley.
  - 2007 - Chinook é provado como um jogo resolvido, garantindo pelo menos um empate com jogo perfeito.
- Xadrez
  - 1997 - Deep Blue da IBM derrota o campeão mundial Garry Kasparov.
  - 2006 - Deep Fritz vence o campeão mundial Vladimir Kramnik.
  - 2017 - AlphaZero da DeepMind aprende xadrez do zero e vence o Stockfish.
- Go
  - 2016 - AlphaGo da DeepMind derrota o campeão mundial Lee Sedol.
  - 2017 - AlphaGo Zero aprende Go do zero e vence a versão anterior.

Tipos de Jogos

- Jogos de Soma Zero
  - O ganho de um agente é a perda do outro.
  - Exemplos: xadrez, damas, jogo da velha.

Tipos de Jogos

- Jogos de Soma Zero
  - O ganho de um agente é a perda do outro.
  - Exemplos: xadrez, damas, jogo da velha.
- Jogos de Soma Não-Zero
  - Os agentes podem cooperar ou competir, e ambos podem ganhar ou perder.
  - Exemplos: negociação, leilões.

Tipos de Jogos

- Jogos de Soma Zero
  - O ganho de um agente é a perda do outro.
  - Exemplos: xadrez, damas, jogo da velha.
- Jogos de Soma Não-Zero
  - Os agentes podem cooperar ou competir, e ambos podem ganhar ou perder.
  - Exemplos: negociação, leilões.
- Jogos Cooperativos
  - Agentes trabalham juntos para alcançar um objetivo comum.
  - Exemplos: jogos em equipe, alianças estratégicas.

Jogos de Soma Zero

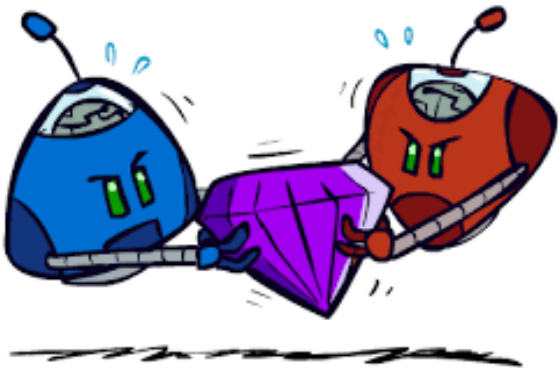


Figure: Jogo de soma zero.

Jogos de Soma Zero

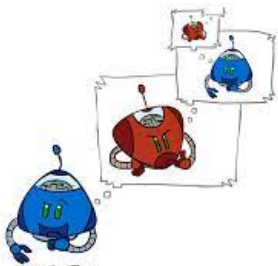
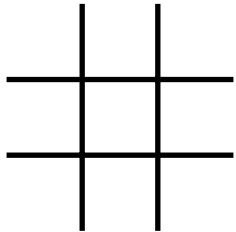


Figure: Jogo de soma zero.

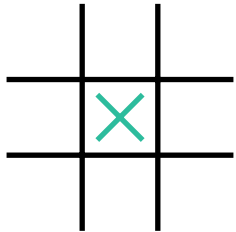
- Um agente está tentando descobrir o que fazer para maximizar sua utilidade.
- Como decidir? Ele pensa nas consequências das possíveis ações.
- Ele precisa pensar no seu oponente também...
- O oponente também está pensando no que fazer, etc.
- Cada um imaginará qual seria a resposta do oponente às suas ações.

Busca Competitiva



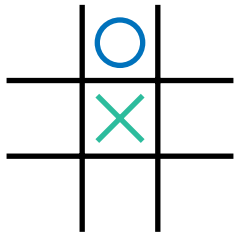
- Dois jogadores (X e O) se revezam para marcar espaços em uma grade 3x3.
- O objetivo é alinhar três de seus símbolos horizontalmente, verticalmente ou diagonalmente.
- O jogo termina em vitória para um jogador ou em empate se todos os espaços forem preenchidos sem um vencedor.

Busca Competitiva



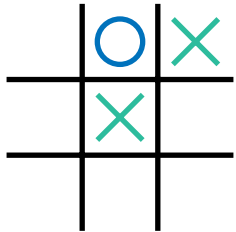
- Dois jogadores (X e O) se revezam para marcar espaços em uma grade 3x3.
- O objetivo é alinhar três de seus símbolos horizontalmente, verticalmente ou diagonalmente.
- O jogo termina em vitória para um jogador ou em empate se todos os espaços forem preenchidos sem um vencedor.

Busca Competitiva



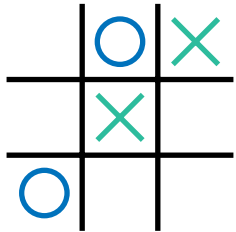
- Dois jogadores (X e O) se revezam para marcar espaços em uma grade 3x3.
- O objetivo é alinhar três de seus símbolos horizontalmente, verticalmente ou diagonalmente.
- O jogo termina em vitória para um jogador ou em empate se todos os espaços forem preenchidos sem um vencedor.

Busca Competitiva



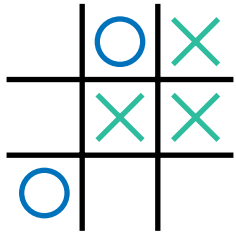
- Dois jogadores (X e O) se revezam para marcar espaços em uma grade 3x3.
- O objetivo é alinhar três de seus símbolos horizontalmente, verticalmente ou diagonalmente.
- O jogo termina em vitória para um jogador ou em empate se todos os espaços forem preenchidos sem um vencedor.

Busca Competitiva



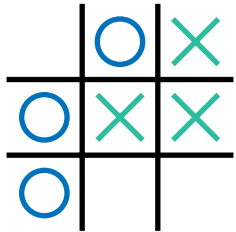
- Dois jogadores (X e O) se revezam para marcar espaços em uma grade 3x3.
- O objetivo é alinhar três de seus símbolos horizontalmente, verticalmente ou diagonalmente.
- O jogo termina em vitória para um jogador ou em empate se todos os espaços forem preenchidos sem um vencedor.

Busca Competitiva



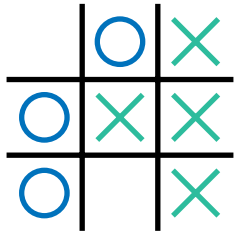
- Dois jogadores (X e O) se revezam para marcar espaços em uma grade 3x3.
- O objetivo é alinhar três de seus símbolos horizontalmente, verticalmente ou diagonalmente.
- O jogo termina em vitória para um jogador ou em empate se todos os espaços forem preenchidos sem um vencedor.

Busca Competitiva



- Dois jogadores (X e O) se revezam para marcar espaços em uma grade 3x3.
- O objetivo é alinhar três de seus símbolos horizontalmente, verticalmente ou diagonalmente.
- O jogo termina em vitória para um jogador ou em empate se todos os espaços forem preenchidos sem um vencedor.

Busca Competitiva



- Dois jogadores (X e O) se revezam para marcar espaços em uma grade 3x3.
- O objetivo é alinhar três de seus símbolos horizontalmente, verticalmente ou diagonalmente.
- O jogo termina em vitória para um jogador ou em empate se todos os espaços forem preenchidos sem um vencedor.

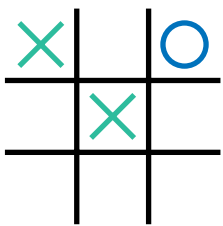
Formalização de um jogo

Um jogo adversarial pode ser descrito por:

$$\boxed{\langle S, s_0, \text{Player}, \text{Actions}, \text{Result}, \text{Terminal}, \text{Utility} \rangle}$$

- S: conjunto de estados do jogo;
- s_0: estado inicial;
- $\text{Player}(s)$: jogador que deve agir no estado s;
- $\text{Actions}(s)$: ações legais no estado s;
- $\text{Result}(s, a)$: estado resultante da ação a;
- $\text{Terminal}(s)$: indica se o jogo terminou;
- $\text{Utility}(s, p)$: utilidade terminal para o jogador p.

Exemplo de formalização no jogo da velha



- $\text{Player}(s) = O$.
- $\text{Actions}(s) = \{1, 3, 5, 6, 7, 8\}$.
- Se $a = 8$:

$$\text{Result}(s, 8) = s'$$

  onde O ocupa a casa 8.
- $\text{Terminal}(s) = false$.

Minimax

- Minimax é um algoritmo usado em jogos de soma zero para determinar a melhor jogada para um jogador, assumindo que o oponente também joga de forma ótima.
- O algoritmo constrói uma árvore de decisão, onde cada nó representa um estado do jogo e cada aresta representa uma ação possível.
- O objetivo do jogador "Max" é maximizar sua pontuação, enquanto o jogador "Min" tenta minimizar a pontuação de "Max".
- O valor de cada nó é calculado recursivamente, propagando os valores dos nós folha para cima na árvore.

Recorrência do Minimax

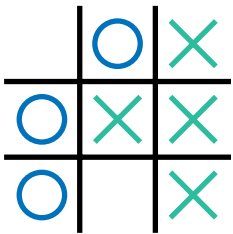
O valor de um estado é definido recursivamente:

$$V(s) = \begin{cases} \text{Utility}(s), & \text{Terminal}(s), \\ \max\limits_{a \in \text{Actions}(s)} V(\text{Result}(s,a)), & \text{Player}(s) = MAX, \\ \min\limits_{a \in \text{Actions}(s)} V(\text{Result}(s,a)), & \text{Player}(s) = MIN. \end{cases}$$

A melhor ação para MAX é:

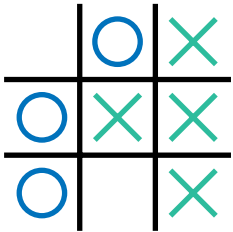
$$\boxed{a^* = \arg \max_{a \in \text{Actions}(s)} V(\text{Result}(s,a))}$$

Minimax - Exemplificação

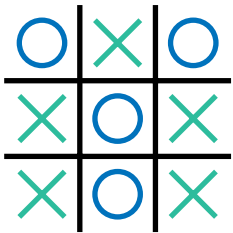


Valor: $+1$

Minimax - Exemplificação

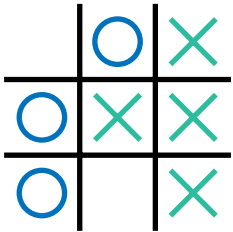


Valor: $+1$

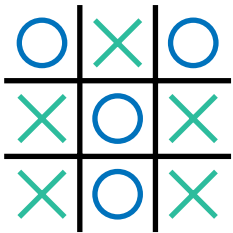


Valor: 0

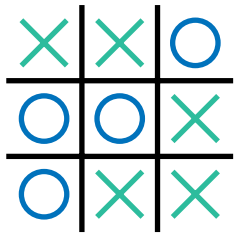
Minimax - Exemplificação



Valor: $+1$



Valor: 0



Valor: -1

Minimax

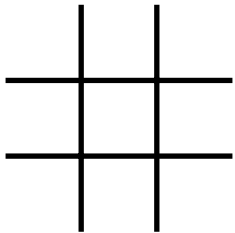
- $MAX(X)$: Jogador que tenta maximizar a pontuação.
- $MIN(O)$: Jogador que tenta minimizar a pontuação de MAX.

Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.

Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.

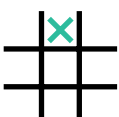


Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.

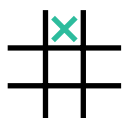
Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.

player()

Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.

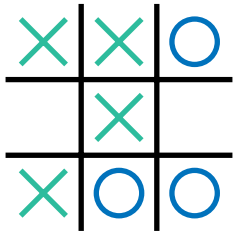
player() = 0

Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.

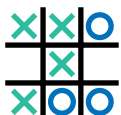
Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.



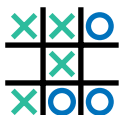
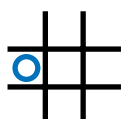
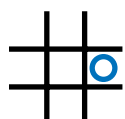
Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.
- Modelo de Transição ($result(s, a)$): Função que determina o próximo estado após a aplicação de uma ação.

actions()

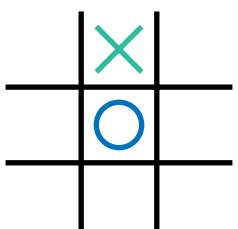
Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.
- Modelo de Transição ($result(s, a)$): Função que determina o próximo estado após a aplicação de uma ação.

actions() = {  ,  }

Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.
- Modelo de Transição ($result(s, a)$): Função que determina o próximo estado após a aplicação de uma ação.



Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.
- Modelo de Transição ($result(s, a)$): Função que determina o próximo estado após a aplicação de uma ação.

result( , ) =

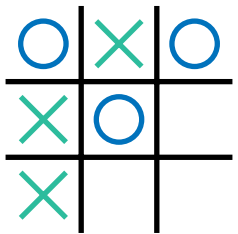
Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.
- Modelo de Transição ($result(s, a)$): Função que determina o próximo estado após a aplicação de uma ação.

result( , ) = 

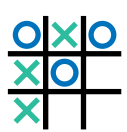
Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.
- Modelo de Transição ($result(s, a)$): Função que determina o próximo estado após a aplicação de uma ação.
- Teste de Término ($terminal(s)$): Verifica se o jogo terminou.



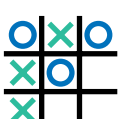
Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.
- Modelo de Transição ($result(s, a)$): Função que determina o próximo estado após a aplicação de uma ação.
- Teste de Término ($terminal(s)$): Verifica se o jogo terminou.

terminal() =

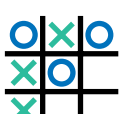
Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.
- Modelo de Transição ($result(s, a)$): Função que determina o próximo estado após a aplicação de uma ação.
- Teste de Término ($terminal(s)$): Verifica se o jogo terminou.

terminal() = false

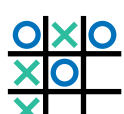
Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.
- Modelo de Transição ($result(s, a)$): Função que determina o próximo estado após a aplicação de uma ação.
- Teste de Término ($terminal(s)$): Verifica se o jogo terminou.

terminal() = false terminal() =

Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.
- Modelo de Transição ($result(s, a)$): Função que determina o próximo estado após a aplicação de uma ação.
- Teste de Término ($terminal(s)$): Verifica se o jogo terminou.

terminal() = false terminal() = true

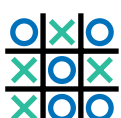
Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.
- Modelo de Transição ($result(s, a)$): Função que determina o próximo estado após a aplicação de uma ação.
- Teste de Término ($terminal(s)$): Verifica se o jogo terminou.
- Função de Utilidade ($utility(s, p)$): Atribui um valor numérico a um estado terminal para um jogador específico.



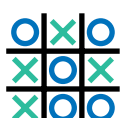
Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.
- Modelo de Transição ($result(s, a)$): Função que determina o próximo estado após a aplicação de uma ação.
- Teste de Término ($terminal(s)$): Verifica se o jogo terminou.
- Função de Utilidade ($utility(s, p)$): Atribui um valor numérico a um estado terminal para um jogador específico.

utility() =

Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.
- Modelo de Transição ($result(s, a)$): Função que determina o próximo estado após a aplicação de uma ação.
- Teste de Término ($terminal(s)$): Verifica se o jogo terminou.
- Função de Utilidade ($utility(s, p)$): Atribui um valor numérico a um estado terminal para um jogador específico.

utility() = - 1

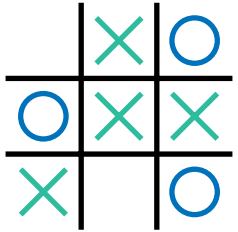
Minimax - Formalização

- Estado Inicial (s_0): Representa a configuração inicial do jogo.
- Jogador ($player(s)$): Indica qual jogador tem a vez de jogar em um estado específico.
- Ações ($actions(s)$): Conjunto de movimentos possíveis para cada jogador.
- Modelo de Transição ($result(s, a)$): Função que determina o próximo estado após a aplicação de uma ação.
- Teste de Término ($terminal(s)$): Verifica se o jogo terminou.
- Função de Utilidade ($utility(s, p)$): Atribui um valor numérico a um estado terminal para um jogador específico.

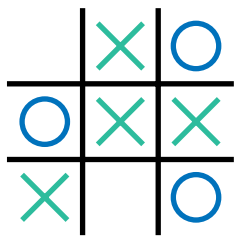
utility() = - 1

utility() = 1

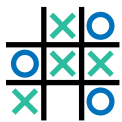
Minimax - Decisões Ótimas

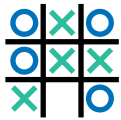


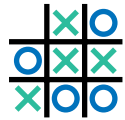
Minimax - Decisões Ótimas

player() = O

Minimax - Decisões Ótimas

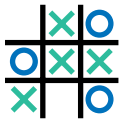
Min-value: 

Max-value: 

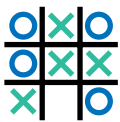
Max-value: 

Minimax - Decisões Ótimas

Min-value:



Max-value:



Max-value:



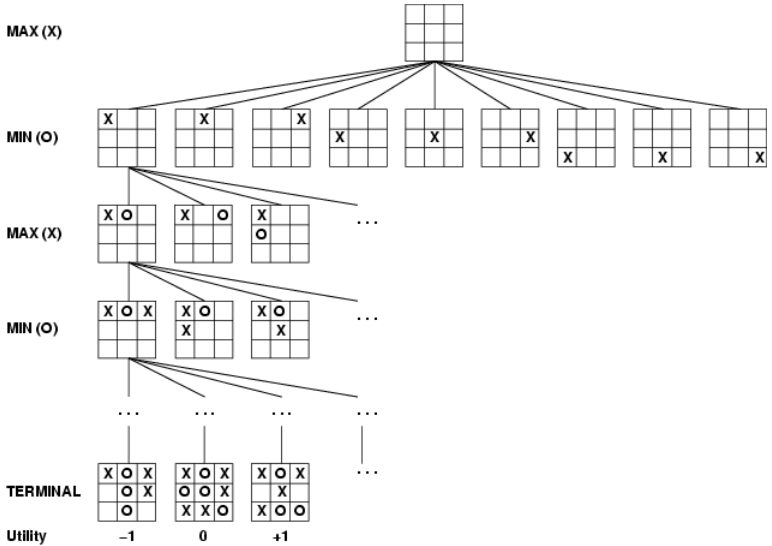
value: 1



value: 0



Minimax - Árvore



Minimax - Decisões Ótimas

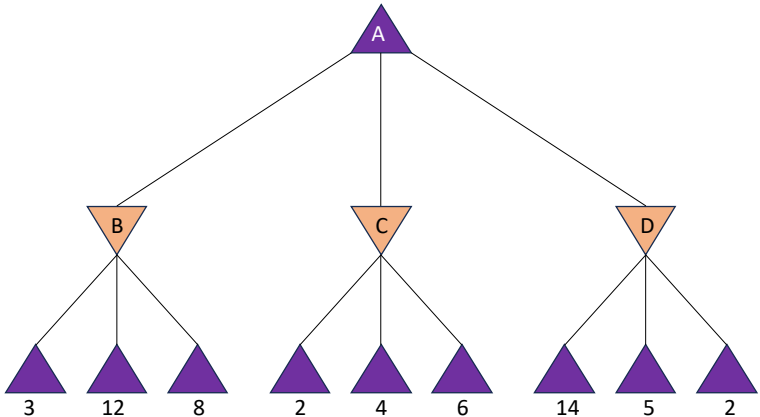


Figure: Árvore de decisão do Minimax com valores.

Minimax - Decisões Ótimas

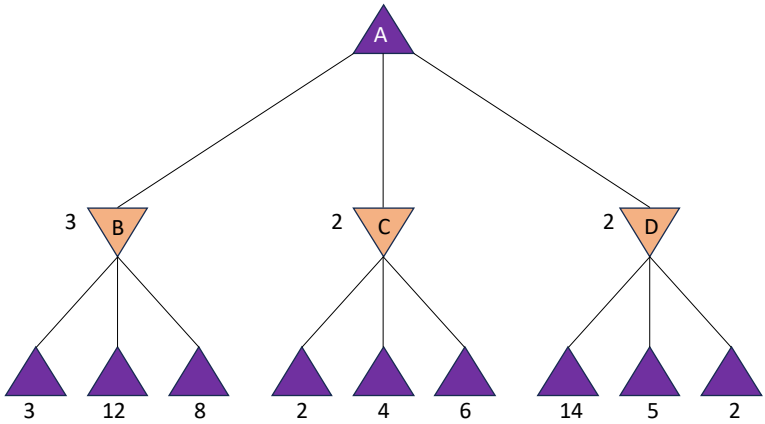


Figure: Árvore de decisão do Minimax com valores.

Minimax - Decisões Ótimas

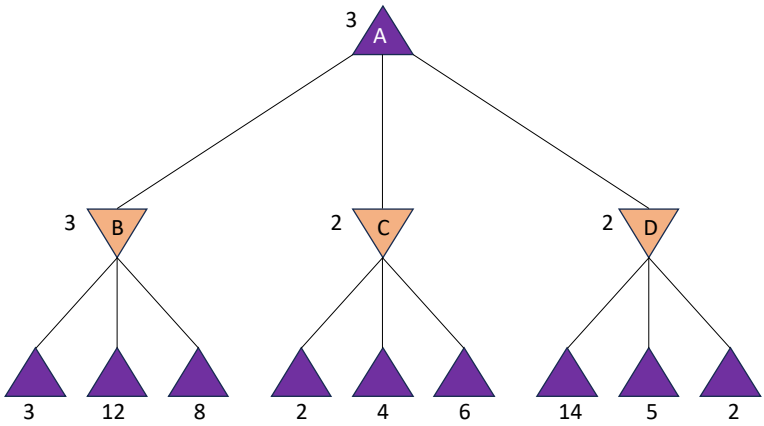
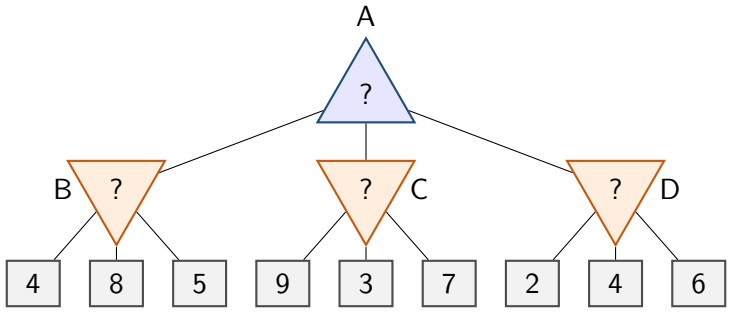


Figure: Árvore de decisão do Minimax com valores.

Exercício rápido: calcule Minimax



Tarefa

Calcule $V(B)$, $V(C)$, $V(D)$ e $V(A)$.

Minimax

- Dado um estado s:
  - MAX escolhe a ação a que maximiza o valor de $MIN(result(s, a))$.
  - MIN escolhe a ação a que minimiza o valor de $MAX(result(s, a))$.
- O valor de um estado terminal é dado pela função de utilidade.
- O algoritmo explora toda a árvore de decisão, o que pode ser computacionalmente caro.

Minimax - Pseudocódigo

$$\text{MinimaxDecision}(s) = \arg \max_{a \in \text{Actions}(s)} \text{Min-Value}(\text{Result}(s, a))$$

$$\text{Max-Value}(s) = \begin{cases} \text{Utility}(s), & \text{if terminal}(s) \\ \max_{a \in \text{Actions}(s)} \text{Min-Value}(\text{Result}(s, a)), & \text{otherwise} \end{cases}$$

$$\text{Min-Value}(s) = \begin{cases} \text{Utility}(s), & \text{if terminal}(s) \\ \min_{a \in \text{Actions}(s)} \text{Max-Value}(\text{Result}(s, a)), & \text{otherwise} \end{cases}$$

Complexidade do Minimax

Se:

- b é o fator de ramificação;
- m é a profundidade máxima;

então a árvore completa possui, no pior caso:

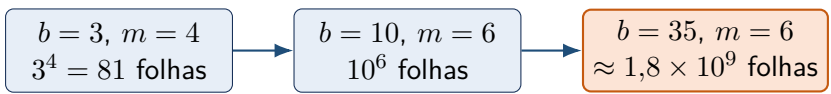
$$1 + b + b^2 + \cdots + b^m = O(b^m)$$

Consequência

Minimax completo é inviável para jogos grandes.

Precisamos reduzir a busca ou parar antes das folhas terminais.

Explosão combinatória



Resultado

Mesmo com computadores rápidos, aumentar a profundidade em jogos reais é caro. Por isso, engines combinam busca, poda, avaliação e ordenação de movimentos.

Utility versus Evaluation Function

Utility

Valor verdadeiro de um estado terminal:

$$\text{Utility}(s) = \begin{cases} +1 & \textit{vitória} \\ 0 & \textit{empate} \\ -1 & \textit{derrota} \end{cases}$$

Evaluation Function

Estimativa para estado não terminal:

$$\text{Eval}(s) \approx V(s)$$

Exemplo em xadrez:

$$\text{Eval}(s) = \text{material} + \text{mobilidade} + \cdots$$

Utility é exata; Eval é uma aproximação.

Minimax com profundidade limitada

Em jogos grandes, paramos antes dos estados terminais:

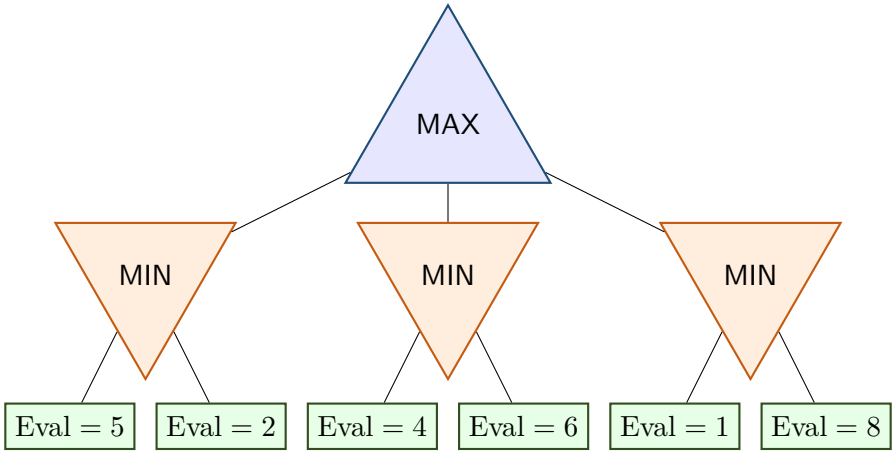
$$Cutoff(s, d) = \text{Terminal}(s) \vee d = 0$$

$$V(s, d) = \begin{cases} \text{Utility}(s), & \text{Terminal}(s), \\ \text{Eval}(s), & d = 0, \\ \max\limits_{a} V(\text{Result}(s, a), d - 1), & \text{Player}(s) = MAX, \\ \min\limits_{a} V(\text{Result}(s, a), d - 1), & \text{Player}(s) = MIN. \end{cases}$$

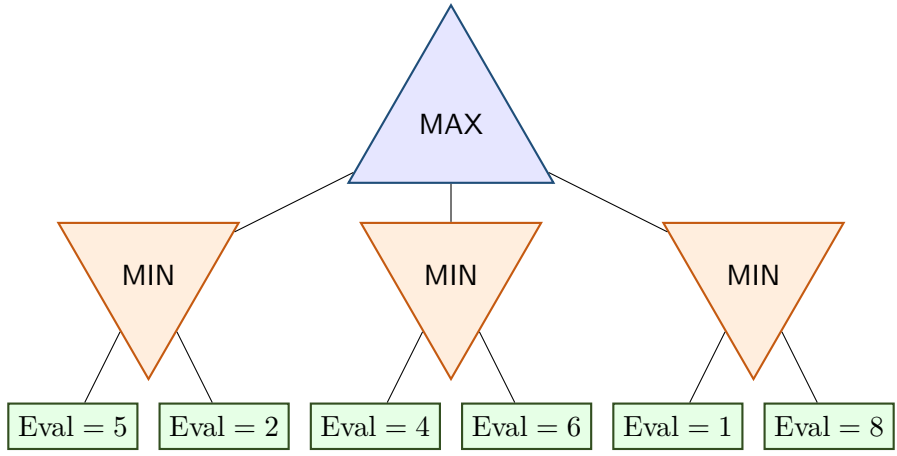
Trade-off

Maior profundidade melhora a decisão, mas aumenta exponencialmente o custo.

Exemplo: folhas de corte



Exemplo: folhas de corte



$$V = \max(\min(5, 2), \min(4, 6), \min(1, 8)) = \max(2, 4, 1) = \boxed{4}$$

Quando Minimax deixa de ser ótimo?

Minimax completo

Se chega aos estados terminais e as hipóteses são satisfeitas, escolhe a ação ótima contra um oponente ótimo.

Minimax limitado

Quando usa $\mathrm{Eval}(s)$, a decisão depende da qualidade da função de avaliação.

Busca limitada troca exatidão por viabilidade computacional.

Minimax - Otimização

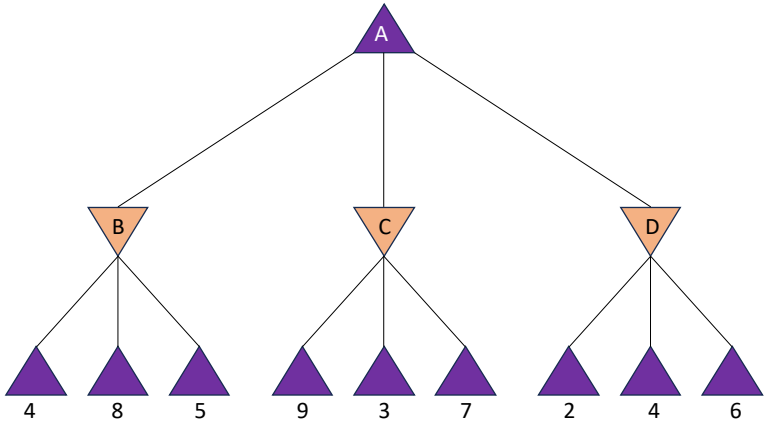


Figure: Árvore de decisão do Minimax com valores.

Minimax - Otimização

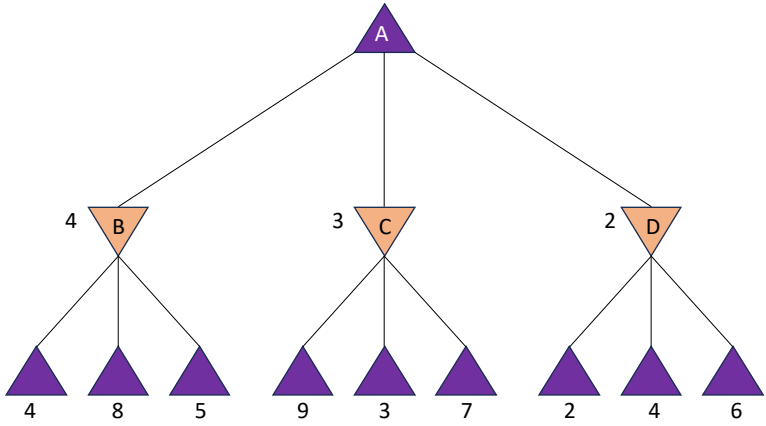


Figure: Árvore de decisão do Minimax com valores.

Minimax - Otimização

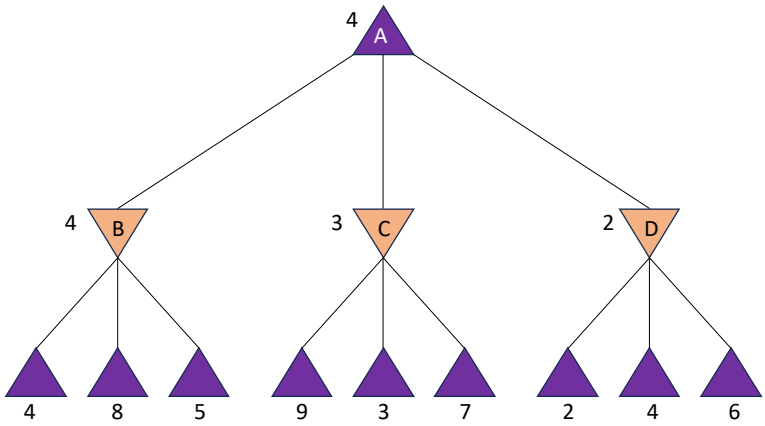


Figure: Árvore de decisão do Minimax com valores.

Minimax - Otimização

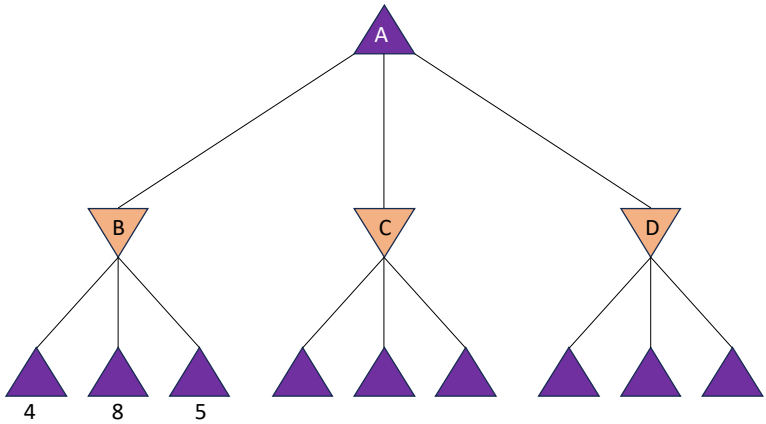


Figure: Árvore de decisão do Minimax com valores.

Minimax - Otimização

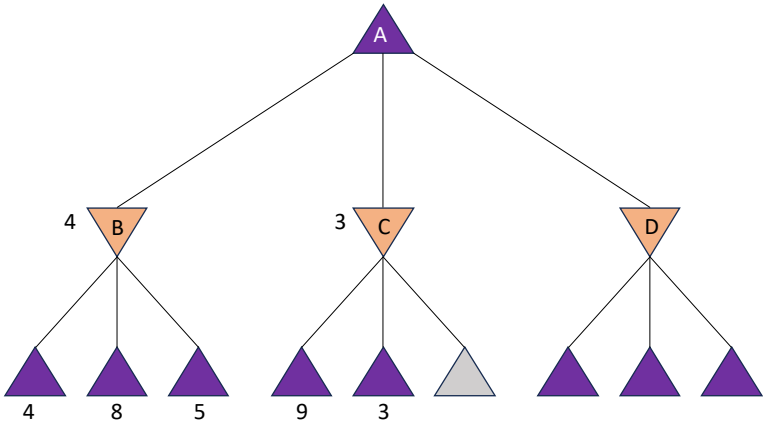


Figure: Árvore de decisão do Minimax com valores.

Minimax - Otimização

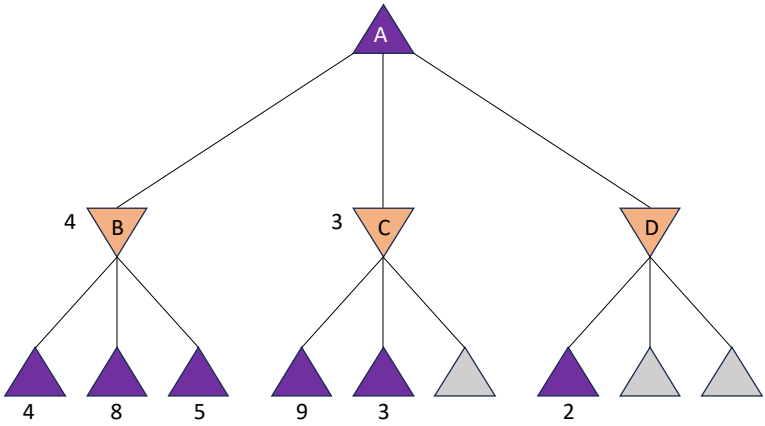


Figure: Árvore de decisão do Minimax com valores.

Minimax - Otimização

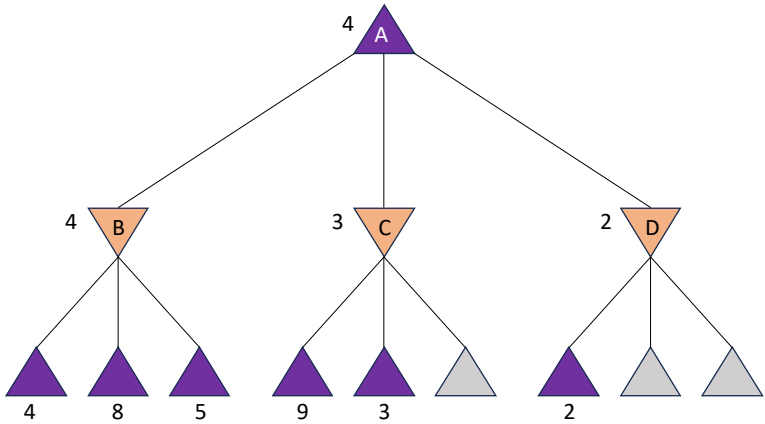


Figure: Árvore de decisão do Minimax com valores.

Minimax - Poda Alpha-Beta

- A poda Alpha-Beta é uma otimização do algoritmo Minimax que reduz o número de nós avaliados na árvore de decisão.
- Introduz dois valores, Alpha e Beta, que representam os limites inferior e superior para a avaliação dos nós.
- Alpha é o valor máximo que o jogador Max pode garantir até aquele ponto, enquanto Beta é o valor mínimo que o jogador Min pode garantir.
- Se em algum ponto Beta for menor ou igual a Alpha, a exploração daquele ramo pode ser interrompida (podada), pois não afetará a decisão final.

Ordenação de movimentos importa

Pior ordem

Pouca ou nenhuma poda.

$$O(b^m)$$

Ordem ideal

Muita poda.

$$O(b^{m/2})$$

Interpretação

Com boa ordenação de movimentos, a mesma quantidade de tempo pode permitir quase dobrar a profundidade pesquisada.

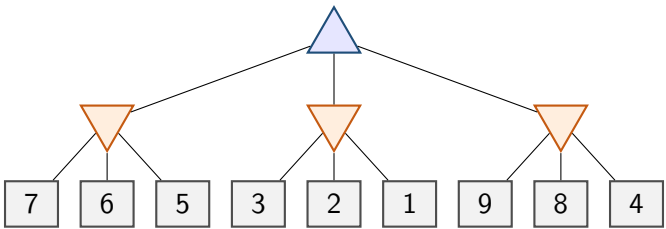
O valor final é igual; o número de nós avaliados muda.

Atividade: duas ordens, mesma árvore

Tarefa

Aplique α-β na mesma árvore em duas ordens:

1. filhos da esquerda para a direita;
2. filhos da direita para a esquerda.

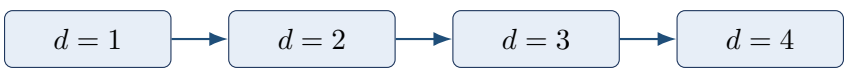


Conte quantas folhas são avaliadas em cada caso.

Iterative Deepening

Quando existe limite de tempo, usamos profundidades crescentes:

$$d = 1, 2, 3, 4, \ldots$$



- sempre temos uma melhor jogada corrente;
- profundidades anteriores ajudam a ordenar movimentos;
- é a abordagem natural em jogos com tempo limitado.

O que acontece se o adversário não for ótimo?

Minimax modela MIN como adversário ótimo:

$$V(s) = \min_a V(\text{Result}(s, a)).$$

Mas alguns ambientes envolvem:

- oponentes aleatórios;
- decisões estocásticas;
- dados ou cartas;
- jogadores humanos imperfeitos.

Nesses casos, o pior caso pode ser conservador demais.

Expectimax

Em vez de assumir que o outro agente escolhe o pior para MAX, podemos modelar incerteza por valor esperado.

$$V(s) = \begin{cases} \text{Utility}(s), & \text{Terminal}(s), \\ \max_a V(\text{Result}(s,a)), & \text{Player}(s) = MAX, \\ \sum_a P(a \mid s) V(\text{Result}(s,a)), & \text{Player}(s) = CHANCE. \end{cases}$$

Interpretação

Minimax decide contra o pior caso. Expectimax decide considerando uma distribuição de probabilidades.

Minimax versus Expectimax

Se o nó é MIN

$$A \to \{3,5\} \quad \Rightarrow \quad V(A) = 3$$

$$B \to \{1,10\} \quad \Rightarrow \quad V(B) = 1$$

Escolha: A.

Se o nó é acaso

$$E[A] = \frac{3+5}{2} = 4$$

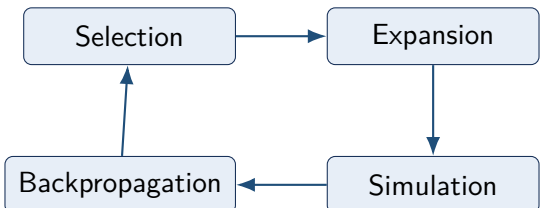
$$E[B] = \frac{1+10}{2} = 5{,}5$$

Escolha: B.

A mesma árvore pode gerar decisões diferentes.

Monte Carlo Tree Search (MCTS)

Quando a árvore é enorme, pode ser melhor amostrar do que explorar sistematicamente.



- usado em jogos com fator de ramificação muito alto;
- concentra simulações nos ramos mais promissores;
- base importante de AlphaGo/AlphaZero.

Minimax, Expectimax e MCTS: comparação

Método	Modelo	Vantagem	Limitação
Minimax	adversário ótimo	robusto contra pior caso	caro e conservador
α-β	adversário ótimo	mesma decisão com menos nós	depende da ordem
Expectimax	acaso/probabilidade	maximiza valor esperado	exige probabilidades
MCTS	amostragem	escala para árvores grandes	resultado aproximado

Minimax - Otimização

- Existem quantas possibilidades para o jogo da velha?

Minimax - Otimização

- Existem quantas possibilidades para o jogo da velha?
  - 255.168
- Existem quantas possibilidades o jogo de xadrez?

Minimax - Otimização

- Existem quantas possibilidades para o jogo da velha?
  - 255.168
- Existem quantas possibilidades o jogo de xadrez?
  - $\approx 10^{120}$
- Existem quantas possibilidades o jogo de Go?

Minimax - Otimização

- Existem quantas possibilidades para o jogo da velha?
  - 255.168
- Existem quantas possibilidades o jogo de xadrez?
  - $\approx 10^{120}$
- Existem quantas possibilidades o jogo de Go?
  - $\approx 10^{170}$